

SYNENT TECHNOLOGY – INTERNSHIP PROJECT REPORT

Task 8: Online Course Platform (MERN)

Project Type: Full-Stack Web Development

Technology Stack: React, Node.js, Express, MongoDB, JWT, Razorpay, Nodemailer

1. Introduction

The Online Course Platform is a full-stack web application developed to provide a complete digital learning experience. The platform allows students to browse courses, enroll in free or paid courses, make test-mode payments through Razorpay, watch video lessons, and track learning progress. An administrative interface allows authorized administrators to manage courses, modules, lessons, users, and enrollments.

2. Project Objective

The main objective of this project is to develop a practical and scalable online learning platform using the MERN stack. The project demonstrates full-stack development concepts including frontend routing, REST API communication, database management, authentication, payment processing, email services, role-based access, course enrollment, and progress tracking.

3. Key Features

Student Features	Registration/login, course browsing, search/filtering, free enrollment, paid enrollment, Razorpay test-mode checkout, M
Admin Features	Admin authentication, course creation, module and lesson management, user viewing, enrollment viewing.
Authentication	JWT-based authentication with bcrypt password hashing.
Payments	Razorpay integration using test mode with server-side payment signature verification.
Email	Nodemailer SMTP support for registration verification, password reset, and enrollment confirmation.
Database	MongoDB with Mongoose models for application data.

4. Technologies Used

- **Frontend:** React 18, Vite, React Router, Axios
- **Backend:** Node.js and Express
- **Database:** MongoDB with Mongoose
- **Authentication:** JWT and bcrypt
- **Payment Gateway:** Razorpay Test Mode
- **Email Service:** Nodemailer with SMTP
- **Video:** HTML5 video player using supplied lesson video URLs

5. System Architecture

The application follows a client-server architecture. The React frontend provides the user interface and communicates with the Express backend through HTTP APIs using Axios. The backend handles authentication, business logic, course and enrollment operations, payment order creation and verification, and email operations. MongoDB stores users, courses, modules, lessons, and enrollment-related data.

Flow: User → React Frontend → Express REST API → Authentication/Business Logic → MongoDB / Razorpay / SMTP

6. Project Structure

```
course-platform/  
■■■ backend/      Express API, MongoDB models, JWT authentication, Razorpay and email  
services  
■■■ frontend/     React application built with Vite  
■■■ README.md     Project documentation
```

7. Backend Implementation

The backend is implemented with Node.js and Express. MongoDB is accessed through Mongoose. JWT is used to authenticate users, while bcrypt is used for password hashing. The backend exposes REST API endpoints for authentication, courses, lessons, enrollments, administration, and payment-related operations. A health-check endpoint is available at GET /api/health.

8. Frontend Implementation

The frontend is built using React 18 and Vite. React Router is used for navigation between application pages, while Axios is used for communication with the backend. Students can browse and filter courses, view course details, enroll, access My Learning, watch lessons, and monitor progress. Administrators can access the admin area to manage course content and view users and enrollments.

9. Authentication and Security

User authentication is implemented using JSON Web Tokens (JWT). Passwords are hashed using bcrypt rather than being stored as plain text. Protected routes and role-based access are used to restrict administrative functionality. Payment verification is performed on the server using Razorpay's signature verification flow.

10. Payment Integration

Paid courses use Razorpay in test mode. When a student selects a paid course, the application creates a payment order and opens the Razorpay Checkout interface. After payment, the backend verifies the payment signature before confirming the enrollment. This allows the payment workflow to be demonstrated without processing a real transaction.

11. Email Functionality

Nodemailer is used with SMTP configuration for registration verification, password reset, and enrollment confirmation emails. SMTP configuration can be supplied through environment variables. If email credentials are unavailable, the application can continue running while email failures are logged to the console.

12. Course Progress Tracking

After enrollment, students can access course lessons and mark lessons as completed. The application updates the student's progress percentage automatically based on completed lessons, providing a simple learning-progress mechanism.

13. Setup and Execution

Backend: Navigate to the backend directory, run `npm install`, create the `.env` file from `.env.example`, configure MongoDB, JWT, Razorpay and SMTP variables, seed sample data with `node seed.js`, and start the server using `npm run dev` or `npm start`. The API runs on port 5000 by default.

Frontend: Navigate to the frontend directory, run `npm install`, configure `VITE_API_URL` and optionally the Razorpay key, then run `npm run dev`. The Vite development server runs on port 5173 by default.

14. Testing Approach

The project documentation defines testing flows for registration/login, seeded student and admin accounts, course browsing, free enrollment, Razorpay test-mode payment, server-side payment verification, lesson access, progress tracking, and administrative course management. A live end-to-end deployment should be tested with the user's own MongoDB, Razorpay test credentials, and SMTP configuration before production use.

15. Results

The completed project provides the core workflow of an online course platform: users can authenticate, browse courses, enroll in free or paid courses, access learning content after enrollment, and track lesson completion. Administrators have separate functionality for managing course content and viewing users and enrollments.

16. Challenges and Learning Outcomes

- Understanding communication between a React frontend and an Express REST API.
- Implementing secure authentication using JWT and bcrypt.
- Designing MongoDB data models and relationships using Mongoose.
- Integrating a third-party payment gateway in test mode and verifying payments server-side.
- Implementing role-based access for student and administrator workflows.
- Connecting SMTP email functionality with application events.
- Managing frontend routing, API requests, authentication state, and learning progress.

17. Future Enhancements

- Deploy frontend and backend to production hosting.
- Add a dedicated video hosting/CDN provider such as Cloudinary or Mux.
- Add course ratings, reviews, and instructor profiles.
- Add certificates after course completion.
- Improve analytics for students and administrators.
- Add stronger automated testing and CI/CD.
- Introduce production payment configuration only after completing security and deployment checks.

18. Conclusion

The Online Course Platform demonstrates a complete full-stack web development workflow using the MERN stack. It combines a React-based user interface with an Express and MongoDB backend and extends the platform with authentication, payment processing, email services, administrative management, and learning-progress tracking. The

project provides practical experience in building and integrating multiple components of a modern web application.

Internship: Syment Technology – Web Development & Design

Task: Task 8 – Online Course Platform (MERN)